The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Paarth Juneja	24BAI10160
Slot	**Date of Submission**
B22	31st March 2026

1. Introduction

Open source software is very important in the modern technology world. Instead of hiding the code behind closed doors, open source lets anyone look at it, learn from it, and fix its problems. This means people from all over the world can work together to build something much better than one single company could make. It is also great for education because students everywhere can study real, professional code for free and understand how things are actually built.

I chose the Linux kernel for my project because it is the absolute foundation of so many things we use today. It runs most of the internet servers, our Android mobile phones, and even supercomputers. I wanted to learn about something that is incredibly powerful but still open for anyone to explore. Learning about the Linux kernel helps me understand exactly how a computer works deep down, underneath all the normal desktop applications.

To show that I understand how to actually use and manage a Linux system, I have also attached five shell scripts to this project. Writing a research report is good, but real world practice is just as important. These scripts demonstrate my practical Linux administration skills. They show that I can automate daily tasks, manage system files, and work comfortably in the command line environment.

2. Choosing My Software

My Choice	Linux Kernel

3. Project Report Structure

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

To understand why the Linux kernel was created, we have to look back at the computer world in the late 1980s and early 1990s. Back then, personal computers were very weak compared to the massive computers used by universities and big corporations. The big institutions used an operating system called Unix. Unix was extremely powerful. It could handle multiple users at the same time and it was very stable. The problem was that Unix was owned by big companies like AT&T. It was incredibly expensive. A normal person or a college student could never afford to buy a Unix license for their home computer.

Linus Torvalds was a computer science student at the University of Helsinki in Finland in 1991. He had just spent a lot of his money to buy a new personal computer. It had an Intel 386 processor, which was a huge step forward for home computers. Linus really wanted to run a Unix system on his new machine so he could practice programming at home. Because commercial Unix was out of the question, he used an alternative called MINIX.

MINIX was a small operating system created by a professor named Andrew Tanenbaum. It was designed to teach students how operating systems worked. The source code was available in a textbook. However, MINIX had a big limitation. Professor Tanenbaum wanted to keep it very simple for his classes. People

all over the world asked him to add new features or make it run faster, but he usually said no. He did not want his teaching tool to become a messy, complicated project.

Linus was very frustrated by these limits. He wanted a system he could explore, modify, and push to the absolute maximum limit of his new 386 processor. He did not set out to change the world or start a massive movement. He just wanted a tool that did what he needed. He started writing a very basic program in assembly language just to connect his home computer to his university network. He needed to read his emails and download files. Soon, he had to write a driver for his disk drive. Then he had to write a file system to save things. Month by month, he realized he was actually building the core of a brand new operating system.

In August 1991, Linus posted a very famous message to an online newsgroup called comp.os.minix. He announced he was making a free operating system, just as a hobby. He explicitly said it would not be big and professional. He uploaded his code to a public server and invited people to look at it. He did not keep it proprietary because he wanted feedback. He knew his code was not perfect, and he wanted other smart programmers to tell him how to fix his mistakes. When other people started downloading it, testing it, and sending him improvements, the project exploded in size. The community realized that building something together openly was much faster than waiting for one company to do it.

A2. The license — what it actually says

The Linux kernel is released under a specific legal document called the GNU General Public License version 2. This is usually just called the GPLv2.

Richard Stallman, the founder of the free software movement, defined four essential freedoms for software.

- Freedom 0 is the freedom to run the program exactly as you wish, for any purpose at all.
- Freedom 1 is the freedom to study how the program works and change it so it does your computing as you wish.
- Freedom 2 is the freedom to redistribute copies so you can help others.
- Freedom 3 is the freedom to distribute copies of your modified versions to others.

Yes, the GPLv2 license on the Linux kernel explicitly grants all four of these freedoms to anyone who downloads it.

A company absolutely cannot take this software, modify it, and sell it without sharing their changes. The GPL has a very clever legal trick built into it called "copyleft." The rule is simple. You can do whatever you want with the code in your own house. But if you compile the code and distribute the final product to the public or to a customer, you are legally forced to also give them the exact source code, including any secret changes you made. This ensures that the software remains free forever. A company cannot steal the community's hard work and lock it behind a closed door.

This is the main difference between GPL and MIT licensing. The MIT license is very short and very loose. It basically says you can do whatever you want with the code, as long as you do not sue the original creator. You can take an MIT project, add your own code to it, and sell it as a completely closed, proprietary product without sharing anything. If I built my own software, I would choose the GPL. I would want to make sure that my hard work benefits everyone forever. If someone wants to use my code to make money, they should be forced to share their improvements back with the rest of the world.

There is a famous saying in the open source world. It is the difference between "free as in free beer" and "free as in freedom." Free beer just means price. Someone buys you a drink at a bar. It

costs you zero dollars, but you have no control over how the beer was made or what is inside it. Free freedom means free speech. It is about liberty. The Linux kernel is both. You can download it for zero dollars, which is free beer. But more importantly, it is free freedom. You have the total liberty to look inside the code, learn from it, rip it apart, and build something new.

A3. The ethics of open source

Open source is not just a way to write code. It is a big ethical philosophy about how human knowledge should be treated.

Should all software be open source? A very strong argument for "yes" is that software is now the foundation of human society. It runs our hospitals, our banks, and our cars. When software is open source, anyone can inspect it for dangerous bugs or security flaws. It also allows students in poor countries to study the same professional code that is used in rich countries. It equalizes education. However, a strong argument for "no" is the reality of economics. Writing complex software takes thousands of hours of intense labor. Programmers have families to feed and bills to pay. If every single piece of software was completely open and free to share, it might be very difficult for small software companies to survive and pay their employees. Some specialized software might never get built if there is no profit motive.

Is it ethical for huge companies to build a profitable product on free community labor? Companies like Red Hat, Google, and Meta make billions of dollars in profit, and their entire business relies on the Linux kernel. This sounds unfair at first, but it is actually highly ethical because of how they behave. These companies do not just take the code. They hire hundreds of the best programmers in the world and pay them full-time salaries just to write new code for the Linux kernel. They share this new code with everyone. It is a mutual relationship. They get a free, stable

operating system, and in return, they fund the development that keeps the project alive. If a company only took the code and refused to ever help fix bugs, the community would consider that highly unethical.

The phrase "standing on the shoulders of giants" means that modern innovation relies on the hard work of people from the past. You do not have to invent the wheel from scratch every time you want to build a car. In the context of software, open source completely enables original innovation. Because the Linux kernel is open and free, a young startup company does not need to spend five years writing a basic operating system just to launch a new smart device. They can take Linux, put it on their device on day one, and immediately start working on their unique, original idea. It accelerates human progress.

Part B — Linux Footprint (Unit 2)

Because my chosen software is the kernel itself, its footprint on the system is very different from a normal desktop application like a media player or a word processor. It is the core manager of the entire computer.

- **Installation method:** The easiest way to install a kernel is through a package manager. On systems like Ubuntu, you use the "apt" command to download a pre-compiled kernel. On systems like Fedora, you use "dnf" or "RPM". However, many advanced users still compile it from source. You download the raw code from kernel.org, run a command called "make menuconfig" to select exactly which hardware drivers you want, and then run the "make" command to build the actual system files.

- **Key directories:** The kernel is not stored in normal application folders. The main compiled kernel file is usually stored in the /boot directory. It is often named "vmlinuz" along with a version number. The kernel also relies on thousands of extra driver modules. These modules are stored in the /lib/modules directory. When the kernel is running, it also creates special virtual directories called /sys and /proc. These are not real folders on the hard drive. They are windows directly into the computer's memory so you can see what the hardware is doing.
- **User and group:** The kernel does not run as a normal Linux user. It does not even run as the "root" administrator. The kernel runs underneath everything in a special hardware state called Ring 0 or Kernel Mode. It is the absolute master of the system. This matters immensely for security. A normal user program runs in Ring 3 (User Mode). If a normal program crashes or tries to do something malicious, the kernel stops it. If users could run code with kernel privileges, they could destroy the computer's memory or bypass all passwords.
- **Service management:** You cannot manage the kernel like a normal service. There is no command like "systemctl restart kernel". The kernel is the thing that runs systemctl. If the kernel stops working, the entire computer freezes immediately. This is called a kernel panic. To switch to a newly installed kernel, you must completely reboot the physical machine so the bootloader can load the new version into memory.
- **Update model:** The open source community is constantly finding security flaws and writing patches. The people who maintain your specific Linux version (like the team at Ubuntu) take these patches and package them into a simple update. You run your system update tool, the new kernel

downloads to your /boot folder, and it waits there until the next time you restart your computer.

Part C — The FOSS Ecosystem (Units 3 & 4)

The Linux kernel does not exist in isolation. A kernel by itself is just a bridge between hardware and software. Without other tools, a human cannot actually use it.

- **Dependencies:** The Linux kernel heavily depends on tools created by the GNU Project. To compile the kernel, you absolutely must have a compiler. It relies on the GNU C Compiler (GCC). Once the kernel boots up, it needs a way to let a user type commands. It depends on a shell, usually the GNU Bash shell. It also requires a bootloader program, most commonly GRUB, to wake it up when the computer is turned on.
- **Enabled or inspired:** The Linux kernel has inspired and enabled massive amounts of modern technology. The entire Android operating system for mobile phones is built right on top of the Linux kernel. The entire concept of cloud computing and Docker containers only exists because of specific resource management features built into the Linux kernel. It also runs on most of the world's fastest supercomputers.
- **LAMP stack relation:** In the famous LAMP stack (Linux, Apache, MySQL, PHP), Linux is the "L". It is the fundamental bottom layer. Linux is the operating system that manages the network card so Apache can send web pages. It manages the hard drive so MySQL can save data. Without the Linux foundation, the rest of the web stack would have nowhere to exist.

- **Community governance:** The community around the Linux kernel is one of the largest in the world. The primary place where work happens is the Linux Kernel Mailing List. This is a very busy email group where developers submit patches for review. There are also many forums and global conferences. The governance is a strict hierarchy. Thousands of developers submit code to sub-maintainers. Those maintainers pass the best code up to higher level maintainers. At the very top is Linus Torvalds, who still has the ultimate authority to accept or reject code into the official main kernel.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source (Linux Kernel)	Proprietary Alternative (Windows NT Kernel)
Cost	Completely free to download, use, and share across millions of computers.	Requires paying money for a commercial license for every computer.
Security (who can audit the code?)	Anyone in the world can read the code. Thousands of security researchers look for bugs every day.	Closed and secret. Only Microsoft employees are legally allowed to see the source code.

Support and reliability	Huge community forums for free help. You can also buy expensive, reliable enterprise support from companies like Red Hat or Canonical.	Official professional support comes directly from Microsoft and its certified partners.
Freedom to modify	Total freedom. You can change any line of code to make the system fit your exact unique needs.	No freedom. It is illegal and technically impossible to change the core Windows code.
Community vs corporate control	Guided by a massive community of individuals and multiple competing corporations who have to cooperate.	Controlled strictly by one single corporation that makes all the final decisions.
Your overall verdict	Being free of cost and flexible, the Linux kernel too is widely used as anyone can mould it according to their needs	While structured support exists, it completely lacks the freedom that open source might provide

Given everything I have learned, I would absolutely recommend the Linux kernel for a real-world deployment, especially for servers and business infrastructure. It is incredibly stable, highly secure, and very cost-effective. The greatest benefit is that you are never locked in. If a proprietary company decides to raise their prices or stop supporting an old product, you are trapped.

With Linux, because the code is open, you always have the freedom to hire someone to fix your problems or maintain your systems yourself. It gives the user true ownership over their technology.

I would very much like to contribute to it in the future. Writing complex C code for the deepest parts of the kernel is very difficult, and I am not ready to do that yet as a student. However, the ecosystem needs much more than just core code. I can contribute by downloading test versions of the kernel and reporting bugs if my laptop's Wi-Fi stops working. I can also help by writing better documentation or answering questions on community forums for new users. Contributing to open source is about being part of the community, no matter your skill level.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

```
# --- Variables ---
STUDENT_NAME="Paarth Juneja"
SOFTWARE_CHOICE="Linux Kernel"
# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date "+%d-%m-%Y %H:%M:%S")
# --- Display ---
echo "================================"
echo "  Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Name    : $USER_NAME"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
echo "Date    : $DATE"
echo "License : Linux follows GPL"
```

```
paarth@ideapad-5i:~$ ./script1.sh
================================
  Open Source Audit — Paarth Juneja
================================
Name    : paarth
Kernel  : 6.6.87.2-microsoft-standard-WSL2
User    : paarth
Uptime  : up 5 minutes
Date    : 31-03-2026 14:05:22
License : Linux follows GPL
paarth@ideapad-5i:~$
```

Script 2 — FOSS Package Inspector (Unit 2)

```
#!/bin/bash

# Script 2: FOSS Package Inspector

PACKAGE="git"
```

```
# Check if package is installed

if command -v $PACKAGE >/dev/null 2>&1; then

        echo "$PACKAGE is installed."

        echo "Version: $($PACKAGE --version)"

else

        echo "$PACKAGE is NOT installed."

fi

case $PACKAGE in

        httpd)  echo "Apache: the web server that built the open
internet" ;;

        mysql)  echo "MySQL: open source at the heart of millions of
apps" ;;

        git) echo "Git enables distributed version control and
collaboration." ;;

        vlc) echo "VLC plays multimedia files." ;;

        firefox) echo "Firefox supports open web browsing." ;;

        linux) echo "Linux is a kernel which runs servers." ;;

esac
```

```
paarth@ideapad-5i:~$ ./script2.sh
git is installed.
Version: git version 2.43.0
Git enables distributed version control and collaboration.
paarth@ideapad-5i:~$
```

Script 3 — Disk and Permission Auditor (Unit 2)

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")


echo "Directory Audit Report"

echo "----------------------"

for DIR in "${DIRS[@]}"; do

        if [ -d "$DIR" ]; then
```

```
                PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
                SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
                echo "$DIR => Permissions: $PERMS | Size: $SIZE"
        else
                echo "$DIR does not exist on this system"
        fi
done
echo "Config Check"
echo "---------------------"
if [ -f "/boot/config-$(uname -r)" ]; then
        echo "Linux config file found:"
        ls -l "/boot/config-$(uname -r)"
else
        echo "No Linux config file found"
```

```
paarth@ideapad-5i:~$ sudo bash script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 586M
/home => Permissions: drwxr-xr-x root root | Size: 235M
/usr/bin => Permissions: drwxr-xr-x root root | Size: 175M
/tmp => Permissions: drwxrwxrwt root root | Size: 32K
Config Check
---------------------
No Linux config file found
paarth@ideapad-5i:~$
```

Script 4 — Log File Analyzer (Units 2 & 5)

```
!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0
if [ ! -f "$LOGFILE" ]; then
        echo "Error: File $LOGFILE not found."
        exit 1
fi
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
```

```
            COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

```
paarth@ideapad-5i:~$ ./script4.sh script1.sh \#
Keyword '#' found 6 times in script1.sh
Last 5 matches:
# Script 1: System Identity Report
# Author: Paarth Juneja | Course: Open Source Software
# --- Variables ---
# --- System info ---
# --- Display ---
paarth@ideapad-5i:~$
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"
echo "I use $TOOL everyday." >> $OUTPUT
echo "Freedom means $FREEDOM to me." >> $OUTPUT
echo "I want to build $BUILD and share freely." >> $OUTPUT
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

```
paarth@ideapad-5i:~$ ./script5.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Liberty
3. Name one thing you would build and share freely: Devtools
Manifesto saved to manifesto_paarth.txt
I use Git everyday.
Freedom means Liberty to me.
I want to build Devtools and share freely.
paarth@ideapad-5i:~$
```